CS-600: Software Design and Development

9-1 Project: Application Enhancement and Implementation

Malgorzata Debska

Southern New Hampshire University

Matthew Scuteri

June 15, 2026

Sales Guru+ Software Engineering Final Project

Software engineering is a structured discipline that combines technical knowledge with project management, quality assurance, and strategic planning to create reliable and maintainable applications. Successful software projects require developers to understand business needs, translate those needs into technical requirements, and implement solutions that stay scalable as organizations evolve. According to Pressman and Maxim (2020), following a systematic software engineering process reduces development risks, improves software quality, and increases the likelihood of project success. ACME Sales Resources has built its reputation by providing tools that help sales professionals organize customer information and strengthen business relationships. One of its primary products, Sales Guru, has been used successfully for years to manage companies and their sales representatives. As customer expectations have changed, ACME has decided to expand the platform into Sales Guru+, adding the ability to manage products, invoices, and sales transactions while keeping the stability of the existing application. The updated system must also introduce external authentication services and graphical data visualization capabilities to improve security and usability. The purpose of this project is to analyze the requirements for Sales Guru+, recommend a proper software engineering approach, design a maintainable solution, and explain how the new functionality can be implemented using software engineering best practices. In addition to technical implementation, this paper examines testing strategies, performance metrics, sustainability planning, and long-term maintainability to ensure that the application continues to meet business objectives as it evolves.

Part One: Requirements Analysis and Planning

Requirements Analysis and Planning

The success of any software project depends on a thorough understanding of business needs and technical requirements before development begins. Proper requirements analysis reduces ambiguity, minimizes costly redesigns, and helps ensure that the final product satisfies stakeholder expectations (Pressman & Maxim, 2020). In the case of Sales Guru+, the goal is not only to preserve the functionality of the existing Sales Guru application but also to expand it into a more comprehensive sales management system capable of tracking products, invoices, and customer orders.

Functional and Nonfunctional Requirements

The original Sales Guru application provides a reliable platform for managing customer companies and their sales representatives. Its core functionality includes assigning unique customer numbers, generating unique representative IDs, linking representatives to companies, restricting access to authenticated users, encrypting data transmitted across the network, displaying representatives for a selected company, and listing all companies stored in the database. These capabilities have supported the organization for many years and serve as the foundation for the enhanced Sales Guru+ system.

The Sales Guru+ enhancement introduces several new functional requirements that significantly expand the application's capabilities. First, the system must be able to create, store, update, and retrieve invoices within the database. Second, it must maintain a catalog of products or items available for sale. Third, invoice line items must be associated with both invoices and products

so that complete purchase information can be aggregated and displayed accurately. Fourth, user authentication should be managed through an external authentication database rather than relying solely on the existing internal mechanism. Finally, the application should provide graphical visualization of business data, allowing users to review information through dashboards or user-friendly interfaces instead of relying only on text-based displays.

In addition to these functional requirements, several nonfunctional requirements are critical to the success of the project. Security is one of the highest priorities because the application stores sensitive customer and financial information. Communications must remain encrypted; authentication should be reliable and secure, and unauthorized access must be prevented. Performance is also important, as invoice retrieval, reporting, and authentication operations should occur quickly even as the volume of stored data increases. Reliability is essential because organizations depend on the system for daily business operations, making downtime or data corruption unacceptable. The software should also be maintainable, allowing developers to understand, change, and extend the code through modular design, clear documentation, and adherence to proven coding standards. Applying clean architecture principles and separating responsibilities into well-defined components improves long-term maintainability and simplifies future enhancements (Martin, 2018). Finally, scalability should be built into the design so that future capabilities, such as predictive analytics, coordination management, and sales forecasting, can be incorporated without requiring major architectural changes (Sommerville, 2016).

Several notable differences exist between the baseline application and the new Sales Guru+ module. The original system focuses primarily on managing company and representative information, while Sales Guru+ introduces entirely new business entities, including invoices, products, and invoice line items. These additions create more complex relationships within the

database and require added validation rules to preserve data integrity and ensure correct calculations. The authentication mechanism also changes from the existing implementation to an external authentication source, increasing flexibility while requiring secure integration with outside systems. Furthermore, the addition of graphical interfaces transforms the application from a simple contact management tool into a more comprehensive sales management platform that provides users with an improved and more intuitive experience.

From a software engineering perspective, the enhanced design follows best practices by emphasizing modularity, separation of concerns, and reusable components. Organizing the application into distinct layers for presentation, business logic, and data access reduces coupling between components and makes future updates easier to implement. These design principles support long-term sustainability and align with established object-oriented design practices that encourage flexibility and code reuse (Gamma et al., 1994).

Analysis of the Problem Statement

Although the project documentation provides a clear overview of the desired enhancements, several assumptions and potential gaps must be considered before implementation begins. One assumption is that the existing PostgreSQL database infrastructure will continue to be used and can support the other entities without requiring a complete redesign. Another assumption is that historical company and representative records will remain compatible with the updated schema and continue functioning after migration.

Several implementation details are not explicitly defined in the requirements. For example, the documentation does not specify how invoice numbers should be generated or whether they must

follow a sequential format. It also does not describe how deleted products should be handled if they already appear on historical invoices, whether invoices may be edited after submission, or how concurrent updates from multiple users should be managed. Similarly, backup procedures, disaster recovery expectations, audit logging, and user authorization levels beyond authentication are not addressed. Finding these omissions early in the planning process reduces uncertainty and helps developers set up consistent design decisions.

The project also includes several practical constraints. Existing customers rely on the current Sales Guru application, meaning all legacy functionality must remain operational after the new module is introduced. Downtime deployment should be minimized, and any changes must preserve backward compatibility with existing records. Budget and schedule constraints also encourage reusing the current architecture wherever possible rather than rebuilding the entire application. These constraints reinforce the importance of careful planning, incremental implementation, and comprehensive regression testing.

Key Metrics and Performance Indicators

To evaluate whether Sales Guru+ successfully meets its goals, measurable software metrics and key performance indicators (KPIs) should be set up. The baseline application already measures bug density, database timeout rates, feature completion time, and merge request acceptance rates. While these metrics are still valuable, the introduction of invoice processing and inventory management requires more measurements to watch system performance and quality.

Table 1 presents recommended metrics for the enhanced application.

METRIC	TARGET
Bug density	Fewer than 1 defect per 1,000 lines of code
Database timeout rate	Less than 1% of requests
Invoice retrieval time	Under 2 seconds
Authentication response time	Under 1 second
Successful invoice creation rate	Greater than 99 %
Code coverage	At least 80% through automated tests
Sprint task completion	More than 95% completed on schedule

The existing metrics will inevitably be affected by the implementation of the new module. Because the application will have more code and added database operations, claiming the earlier bug density target may become more challenging. Likewise, invoice aggregation introduces more complex queries that could increase database response times if indexing and optimization are not carefully implemented. Authentication through an external provider may also introduce network latency that did not exist in the original application. To accommodate these changes, performance indicators should be expanded rather than replaced. Database performance should measure invoice-related queries independently, so bottlenecks can be found quickly. Feature completion of metrics should distinguish between maintenance work and new functionality to provide a more right assessment of development productivity. Code quality metrics should

continue evaluating merge requests but should also include automated testing coverage and static analysis results to encourage consistent software quality throughout development.

Test Plan and Verification Activities

A comprehensive testing strategy is necessary to verify that Sales Guru+ satisfies both new and existing requirements. Testing should include functional validation, performance evaluation, security verification, and regression testing to ensure previously implemented features continue running correctly. Functional testing will verify that invoices can be created, stored, changed, and retrieved successfully. More tests should confirm that products can be added to the catalog; invoice line items are correctly associated with invoices and products, and invoice totals are accurately aggregated. Authentication testing should verify integration with the external authentication service while rejecting invalid credentials. User interface testing should confirm that graphical displays present exact information and update appropriately when data changes. Performance testing should evaluate the application's behavior under realistic workloads. Simultaneous invoice retrieval requests, large product catalogs, and multiple concurrent users should not produce unacceptable response times or database failures. Stress testing can further decide system behavior when using beyond expected capacity and name potential bottlenecks before deployment. Edge-case testing is equally important. Examples include invoices holding zero-line items, duplicate product entries, invalid quantities, negative pricing values, extremely large invoices, and temporary authentication service outages. These tests help ensure that unexpected conditions do not compromise system reliability or data integrity. Regression testing must also be expanded. Existing tests verifying unique customer numbers, unique representative IDs, secure authentication, encrypted communication, representative retrieval, and company listings should continue to execute after every code change. Other regression scenarios should

confrm that newly introduced invoice and product functionality does not interfere with previously implemented business logic.

Requirements Specification for Stakeholders

The complete Sales Guru+ application should satisfy the following business specifications:

1. The system shall keep unique company records using customer numbers.
2. The system shall support unique sales representative identifiers and associate representatives with companies.
3. The system shall securely authenticate authorized users using an external source.
4. The system shall encrypt data between clients and servers.
5. The system shall create, store, update, and retrieve invoices.
6. The system shall create and keep product records available for sale.
7. The system shall associate invoice line items with invoices and products while calculating correct totals.
8. The system shall provide graphical visualization of business information through an intuitive interface.
9. The system shall preserve compatibility with existing companies and representative records.
10. The application shall keep high availability, strong performance, and secure processing while staying scalable for future enhancements.

Documenting these requirements comprehensively before implementation begins, stakeholders, developers, and testers share a mutual understanding of project goals. This alignment reduces ambiguity, improves communication, and shows a clear foundation for successful software development and long-term maintenance (Pressman & Maxim, 2020).

Part Two: Software Design

Software Design

Designing software involves much more than deciding how to write the code. A well-designed application should be easy to keep, scalable enough to support future growth, and flexible enough to adapt to changing business needs. Selecting the proper software development life cycle (SDLC) method and software architecture is critical because these decisions influence development efficiency, software quality, and long-term sustainability. For the Sales Guru+ project, the design phase focuses on choosing the best development method, finding a proper architecture, planning future expansion, and creating a strategy that ensures the software is still maintainable over time.

Software Development Life Cycle (SDLC) Methodologies

Several SDLC methodologies could be used to develop Sales Guru+, each with its own strengths and limitations.

Waterfall Methodology

The Waterfall model is one of the oldest and most traditional software development approaches. It follows a sequential process in which each phase's requirements of g gathering, design, implementation, testing, deployment, and maintenance are completed before the next phase begins. This theology works well for projects with stable requirements and limited expected changes because planning occurs extensively before development starts. However, Waterfall can become inefficient when requirements evolve during development since revisiting earlier phases may require significant rework (Sommerville, 2016).

Agile Methodology

Agile development emphasizes iterative progress, collaboration, and continuous improvement. By delivering software in short development cycles and incorporating stakeholder feedback, Agile enables teams to respond quickly to changing requirements while reducing project risk (Martin, 2018). Instead of completing the project in one large cycle, development occurs through short iterations called sprints. Each sprint delivers working software that can be reviewed by stakeholders, allowing requirements to be refined throughout the project. Agile encourages communication between developers, testers, and customers while reducing project risk by naming issues early (Pressman & Maxim, 2020).

DevOps Methodology

DevOps extends Agile principles by integrating software development and operations teams through automation, continuous integration, and continuous deployment (CI/CD). Automated testing, infrastructure management, and deployment pipelines allow organizations to release software updates quickly while supporting stability. DevOps is particularly valuable for applications requiring frequent releases and cloud-based deployment environments.

Selected Methodology and Rationale

For the Sales Guru+ project, Agile is the most proper development method. Although the existing Sales Guru application has still been stable for several years, the introduction of invoice management, inventory tracking, graphical visualization, and external authentication significantly increases project complexity. Agile allows developers to implement these features incrementally while gathering stakeholder feedback after each sprint. This iterative approach encourages continuous integration, frequent testing, and early detection of defects, all of which

contribute to higher software quality and improved customer satisfaction (Pressman & Maxim, 2020). Another reason Agile is an excellent choice is the product roadmap for Sales Guru+, which includes planned enhancements such as order forecasting, coordination management, and product popularity analysis. Since these future capabilities may evolve based on customer needs, an iterative methodology provides the flexibility necessary to adapt requirements without disrupting completed work. Agile also supports continuous testing throughout development rather than postponing validation until the end of the project. This reduces the likelihood of major defects reaching production and allows integration problems to be found earlier. Frequent sprint reviews also improve communication between developers and stakeholders, ensuring that the final product aligns with business goals. For these reasons, I would recommend Agile to both the development team and project stakeholders. Its flexibility, emphasis on collaboration, and ability to accommodate evolving requirements make it particularly well-suited for software projects expected to expand over time.

Software Architecture

Choosing the right architecture is equally important because it figures out how components interact and how easily the application can be supported in the future.

Monolithic Architecture

A monolithic architecture combines presentation logic, business rules, and database interactions into a single deployable application. While simple to implement initially, monolithic systems often become difficult to keep as added functionality is introduced because changes in one area may affect unrelated components.

Microservices Architecture

Microservices divide an application into independently deployable services that communicate through APIs. This architecture improves scalability and fault isolation but also introduces more complexity through service orchestration, network communication, and infrastructure management. For relatively small business applications, the overhead of microservices may outweigh the benefits.

Layered Architecture

A layered architecture separates responsibilities into logical tiers, typically including the presentation layer, business logic layer, and data access layer. User interfaces communicate with business services, which process rules and validations before interacting with the database. This separation promotes modularity, maintainability, and code reuse while simplifying debugging and future enhancements.

Selected Architecture and Justification

For Sales Guru+, layered architecture is the right design choice. The existing application already separates data management from business functionality, making it practical to extend rather than redesign the system completely. The presentation layer can support graphical dashboards and user interaction, while the business layer handles invoice calculations, authentication logic, and validation rules. The persistence layer manages communication with the PostgreSQL database and keeps relationships among companies, representatives, invoices, products, and invoice line items. This architecture supports maintainability because modifications within one layer generally do not require changes to unrelated components. For example, replacing the graphical interface or integrating a different authentication provider would have minimal impact on

database operations or business rules. Similarly, future analytics features could be implemented as added service classes without restructuring the entire application.

Design of the New Module

The Sales Guru+ module introduces several new business entities while preserving compatibility with the existing data model.

The primary entities include:

- Company stand for customer organizations.
- Representative, standing for sales personnel associated with companies.
- Product items are available for purchase.
- The invoice stands for completed customer transactions.
- InvoiceLineItem, linking products to invoices while recording quantities and pricing.

The Invoice service will calculate totals automatically by aggregating line items and validating input before data is saved. Authentication services will communicate with an external provider, while the presentation layer will generate graphical displays of invoices and product information. This modular structure supports clear separation of duties and simplifies maintenance. From a sustainability perspective, the design minimizes coupling between components, allowing developers to extend functionality without changing stable portions of the system. Future features named in the roadmap such as predictive sales forecasting, organization optimization, and product popularity analysis can be implemented as added business services that use existing invoice and product data.

Project Plan

The Sales Guru+ project will follow an Agile development life cycle consisting of multiple iterations. Each sprint will focus on delivering a specific set of features while keeping compatibility with existing functionality.

The project will continue through the following stages:

1. **Requirements Analysis:** Review stakeholder needs, existing documentation, and baseline functionality.
2. **Sprint Planning:** Prioritize work items and define sprint goals.
3. **System Design:** Create database modifications, class diagrams, and interface specifications.
4. **Implementation:** Develop Java classes, repositories, and business logic according to coding standards.
5. **Unit Testing:** Verify individual methods and classes using automated tests.
6. **Integration Testing:** Confirm proper interaction between database components, authentication services, and user interfaces.
7. **Quality Assurance:** Execute regression testing and confirm that requirements have been satisfied.
8. **Deployment:** Release approved functionality into production following organizational release procedures.
9. **Maintenance:** Watch performance, resolve defects, and prepare future enhancements.

Several development constraints must be considered throughout the project. Existing customers depend on uninterrupted operation, requiring backward compatibility and careful regression testing. Database migrations must preserve historical information, and deployment schedules

should minimize service interruptions. Resource limitations and project timelines also encourage reuse of existing infrastructure whenever practical.

Sustainability Plan

Long-term sustainability is essential because software continues evolving long after its initial release. To support ongoing success, Sales Guru+ should implement a structured maintenance strategy focused on updates, scalability, and technical debt management.

Updates

Regular software updates should address security vulnerabilities, dependency upgrades, performance improvements, and user-requested enhancements. Version control and automated testing should verify that updates do not introduce regressions before deployment.

Scalability

As transaction volumes increase, the application should continue providing acceptable performance. Database indexing, optimized queries, and modular business services will allow the system to accommodate larger datasets without major architectural changes. The layered design also supports future cloud deployment or distributed processing if organizational needs expand.

Management of Technical Debt

Technical debt accumulates when short-term implementation decisions create long-term maintenance challenges. To reduce technical debt, developers should follow coding standards, conduct peer code reviews, document business logic thoroughly, and schedule periodic refactoring activities. Maintaining high test coverage and eliminating duplicated code will also improve maintainability and reduce future development costs.

Combining Agile development with a layered architecture and proactive sustainability planning, Sales Guru+ will be well positioned to support future innovation while remaining secure, maintainable, and scalable for years to come.

Part Three: Development and Implementation

Development and Implementation

The development and implementation phase transforms the requirements and design decisions into a working software module. For Sales Guru+, this phase focuses on building the invoice, product, and line-item features while preserving the stability of the existing Sales Guru application. The implementation must follow software engineering best practices, including secure coding, readable Java code, modular structure, automated testing, and traceability between requirements and validation activities. According to Pressman and Maxim (2020), software quality is strengthened when development, testing, and documentation occur throughout the life cycle rather than being treated as separate activities at the end of a project.

Development Stages

The Sales Guru+ module will be implemented through the Agile life cycle described in Part Two. Each stage has a specific purpose and produces deliverables that support the next phase of development.

Requirements Review

The first development stage is reviewing the requirements and confirming that the planned implementation matches stakeholder expectations. During this stage, the development team reviews the baseline Sales Guru functionality and the updated Sales Guru+ requirements. The goal is to confirm what must be preserved, what must be added, and what assumptions must be clarified before coding begins. This includes reviewing the existing company and representative features, the current authentication process, and the new requirements for invoices, products, line items, external authentication, and graphical visualization.

Design Preparation

After requirements are reviewed, the team prepares the technical design. This includes defining new database tables, Java classes, service methods, and user interface components. The module will include classes such as Product, Invoice, and InvoiceLineItem, along with service classes responsible for business logic. This design stage also finds how the new module connects to the existing application so that company and representative records can be linked to invoices without breaking the original functionality.

Implementation

During implementation, developers write Java code for the new module using object-oriented programming principles. Code will be organized into clear layers, including model classes, repository or data access classes, service classes, and interface components. The model classes are business objects, the data access layer manages database interaction, and the service layer applies business rules such as validating invoice totals and preventing invalid quantities. This separation of concerns improves readability, maintainability, and testability.

Unit Testing

Unit testing verifies that individual classes and methods work as expected. For example, tests will confirm that invoice totals are calculated correctly, negative quantities are rejected, and product information is stored accurately. Unit tests are important because they help show errors early before multiple components are integrated.

Integration Testing

Integration testing confirms that the new module works correctly with the existing application. This includes confirming that invoices can be linked to existing companies and representatives, that the database stores new records correctly, and that authentication works with the external source. Integration testing is especially important because Sales Guru+ adds new data relationships and external dependencies.

Quality Assurance and Regression Testing

Quality assurance testing confirms that the complete application satisfies project requirements. Regression testing is also needed to ensure that original Sales Guru features still work after the new module is added. Tests for unique customer numbers, representative IDs, authenticated access, encrypted communication, company listings, and representative listings should still pass after implementation.

Deployment and Maintenance

After testing is complete, the module can move into deployment. Deployment should occur according to ACME's release schedule, with final code baselined before production release. After deployment, the team should monitor system performance, review user feedback, track defects, and plan improvements for later releases.

Development and Testing Tools

Several tools are proper for implementing and validating Sales Guru+. The primary programming language will be Java because the project requires the new module to be implemented in Java and because Java supports object-oriented design, strong typing, and maintainable enterprise development. The database platform should remain in PostgreSQL since the existing application already uses it, and the new module can extend the current relational model. Maven may be used to manage dependencies and build processes, while Git should be used for version control. Version control is important because it allows developers to track changes, review code, and return to earlier versions if needed. For testing, JUnit 5 is a suitable library because it supports automated unit testing in Java. JUnit tests can verify calculations, validation rules, and expected behavior for service methods. If database integration tests are included, JDBC or a testing database configuration can be used to confirm that records have persisted correctly. A code editor or integrated development environment such as IntelliJ IDEA, Eclipse, or the Codio environment can be used for development. The new libraries should be selected based on project need, compatibility, security, and maintainability. Libraries should not be added unnecessarily because every dependency increases maintenance responsibility. If a library is introduced, it should be stable, widely used, actively maintained, and compatible with the existing application. For this project, JUnit 5 is justified because automated testing directly supports the validation requirements. Maven is justified because it simplifies build management and dependency control. The development process will integrate with the existing application by extending the current database and application structure instead of replacing it. Existing company and representative functionality will remain in place, while the new product and invoice features will connect through more classes and tables. This approach reduces risk

because it preserves the stable parts of the application while adding new functionality in a controlled and modular way.

Validation Against Requirements

Validation ensures that the developed module satisfies the requirements identified earlier in the project. A traceability matrix connects each requirement to the testing method used to verify it.

Requirement	Implementation Approach	Testing	Rationale
Store invoices in the database	Create an Invoice class and invoice table	Unit and integration testing	Confirms invoices can be created, saved, and retrieved
Store items for sale	Create a Product class and product table	Unit and database tests	Confirms products are stored and available for invoice use
Aggregate invoice line items	Create InvoiceLineItemclass and calculation service	Unit tests for invoice totals	Confirms totals match quantity multiplied by unit price

Use external authentication	Implement authentication service interface	Authentication integration testing	Confirms users are validated through an external source
Visualize lists of data	Add graphical display layer	User interface testing	Confirms users can view invoices, products, and companies
Preserve original company features	Keep existing company logic and add regression tests	Regression testing	Confirms original Sales Guru functions still work
Preserve original representative features	Maintain representative relationship to companies	Regression testing	Confirms representatives remain linked correctly
Encrypt data in transit	Continue HTTPS requirement	Security verification	Confirms communications remain protected

The development meets the requirements because each functional feature has a corresponding class, database relationship, and test method. For example, invoice storage is addressed through

the invoice model and persistence layer, while invoice aggregation is handled through service logic that calculates totals from line items. Authentication is separated into its own service, so the application can integrate with an external database while keeping security logic maintainable. Several added requirements are implied by the solution even though they are not stated in the original problem statement. The system should confirm input before saving records, prevent negative quantities or prices, handle missing products gracefully, and avoid dropping product records that are already attached to historical invoices. The system should also log errors and provide meaningful messages when authentication or database operations fail.

Life Cycle Process Diagram

The overall development process for Sales Guru+ follows an Agile cycle with continuous testing and feedback.


1. Stakeholder Review
Gather business needs
2. Requirements Analysis
Define functional and nonfunctional requirements
3. Project Planning
Schedule tasks and resources
4. Software Design
Design database, architecture, and user interface
5. Java Development
Implement new features
6. Unit Testing
Test individual components
7. Integration & Regression
Verify modules work together
8. Quality Assurance
Validate requirements
9. Deployment
Release application
10. Maintenance
Fix bugs and add improvements
Repeat cycle for future updates

This process supports continuous improvement because feedback from quality assurance, users, and production monitoring can return to future sprint planning. This makes the project easier to support and expand as ACME adds future features.

Conclusion

The development and implementation plan for Sales Guru+ provides a structured approach for building the new module while preserving existing functionality. By using Java for best practices, automated testing, requirement traceability, and a layered architecture, the project can meet current business needs while being still flexible for future growth. The module design supports security, maintainability, scalability, and clear validation against stakeholder requirements. Through disciplined development and continuous testing, Sales Guru+ can successfully evolve from a contact management application into a stronger sales and order management platform.

References

Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill.

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison Wesley.

Martin, R. C. (2018). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.